Python package: Structure, Documentation, Tests

Structure: how do I organize my package?

- Code should be organized in a modular fashion
- I earlier chose to place the BitString class in __init__.py

- What's better?
  create a separate module called bitstring.py
  (at the same directory level as __init__.py)
- Place your BitString class in it
- Comment out/remove the BitString class in __init__.py

- In __init__.py, add the statement
  `from . bitstring import BitString`
- We should now be able to call each class directly from montecarlo

Documentation types

- README documentation - This is displayed on the GitHub page for your project. Should contain a short summary of the project, information on installation and basic usage, and should link out to more complex documentation if you have it.

- End-user documentation - Should explain the goal of your project and explain how to use it. You might choose to include tutorials or text explanations in this documentation.

- Developer documentation - Developer documentation should explain how your project works and how others can contribute to the project.

- API documentation - API documentation is the documentation of the modules and functions in your library. This is the type of documentation which in-code docstrings are used to generate.

Using Sphinx for Documentation

- (Continuing the instructions from 9_Github.pptx)
- After we run `make html` from the docs/ folder, our documentation can be viewed, for example, in the index.html file (open this in your browser) (this can be found in docs/_build/html/)

- Note: Cookiecutter has already populated the index.md file with some commands (which is what generates the output in index.html)

- Try modifying the index.md (or any other .md) file in docs/ and run:

```
make clean
make html
```

Open the index.html file to see your changes reflected

- We can add our custom descriptions/comments to the docs!

Telling users how to install your package

- Let’s tell users what dependencies your package has, as well as how to install it
- Add the following to the `installation.md` file, and save:

````
You will need an environment with the following packages:

* Python 3.11

* NumPy

* Matplotlib

Once you have these packages installed, you can install montecarlo in the same
environment using

```sh

pip install -e .

```

from the top-level montecarlo/ directory.
````

Run:

```
make clean
make html
```

Open the installation.html file (found in docs/_build/html/) in your web browser to see your changes reflected

Telling users how to use your package

- You should have a file named `usage.md` in docs/
- Open the `usage.md` file and add the highlighted text

````
# Usage

To use montecarlo in a project:

```python
import montecarlo


# Define a new configuration instance for a 6-site lattice
conf = montecarlo.BitString(6)
```
````

Run:

```
make clean
make html
```

Open the usage.html file (found in docs/_build/html/) in your web browser to see your changes reflected

Table of Contents Tree

- We will use `toctree` to generate a special section called a Table of Contents that links to other parts of your projects
- Open the `index.md` file and add the following text

````
# Table of Contents
```{toctree}
:maxdepth: 2
:caption: Contents:

installation
usage
api
````

The maxdepth flag defines the depth of the tree

You can create new .md files and add them here based on your needs

Run:

```
make clean
make html
```

Open the index.html file (found in docs/_build/html/) in your web browser to see your changes reflected
```

Generating the API documentation

- We will use Sphinx to generate API document-ation
- Open the `conf.py` file and modify it to add the text on the right
- Do not remove anything from conf.py

```
import os
import sys
# Add the project's src directory to sys.path so Sphinx can import montecarlo.
sys.path.insert(0, os.path.abspath('../src'))
extensions = [
'sphinx.ext.autodoc',
'sphinx.ext.autosummary',
'myst_parser',
]
autosummary_generate = True
autodoc_default_options = {
'members': True,
'imported-members': True,
'undoc-members': True,
'show-inheritance': True,
}
autodoc_mock_imports = ['numpy']
```

except `extensions = ['myst_parser'],` which we are adding back.

Generating API documentation… continued

- We will use Sphinx to generate API document-ation

- Open the `api.md` file and modify it to add the text on the right

- This tells Sphinx to automatically include your docstrings for all classes & methods in the documentation

````
```{eval-rst}
.. automodule:: montecarlo
:members:
:imported-members:
:undoc-members:
:show-inheritance:
```
````

Run:

```
make clean
make html
```

Open the api.html file (found in docs/_build/html/) in your web browser to see your changes reflected

Commit our changes

- Let’s first ensure that our changes are committed to Git

- First, run `git remote -v`
  The output should look like this (with your repository info, not mine)

```
origin  https://github.com/KarunyaShirali/monte_carlo.git (fetch)
origin  https://github.com/KarunyaShirali/monte_carlo.git (push)
```

- Then run:
```
git add .
git commit -m "Documentation"
git push
```

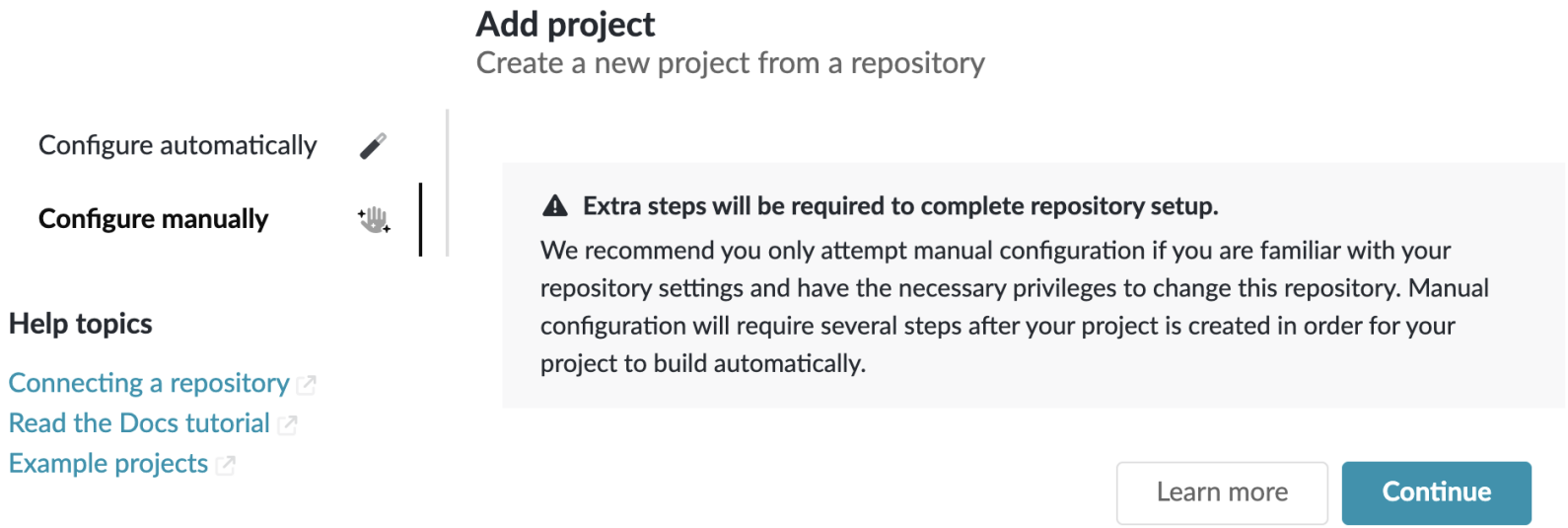
Hosting our documentation: Read the Docs

- We're going to use Read the Docs to host the documentation

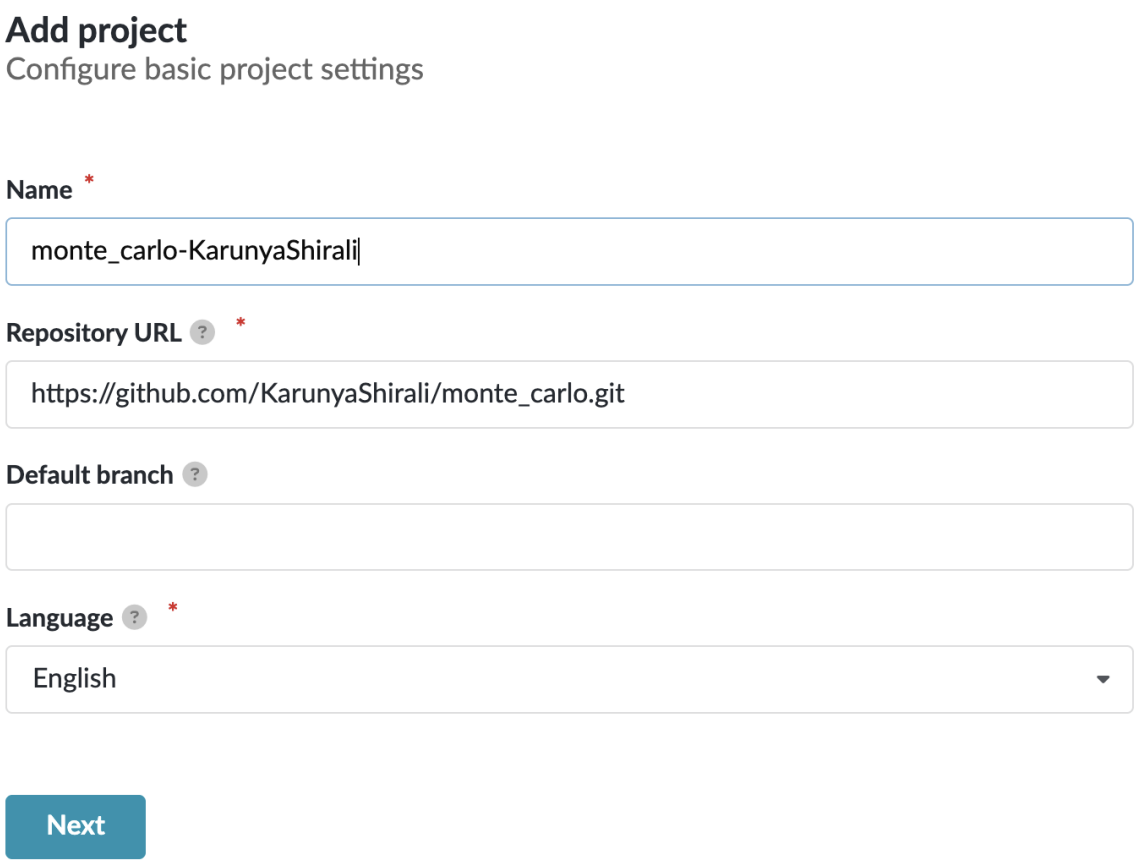
- Go to https://readthedocs.org/ and connect using your GitHub account
  (it will install the Read the Docs community in your GitHub Apps)

- You will need to make your repository Public for these steps

- Select "Add Project"
  and then "Configure manually"

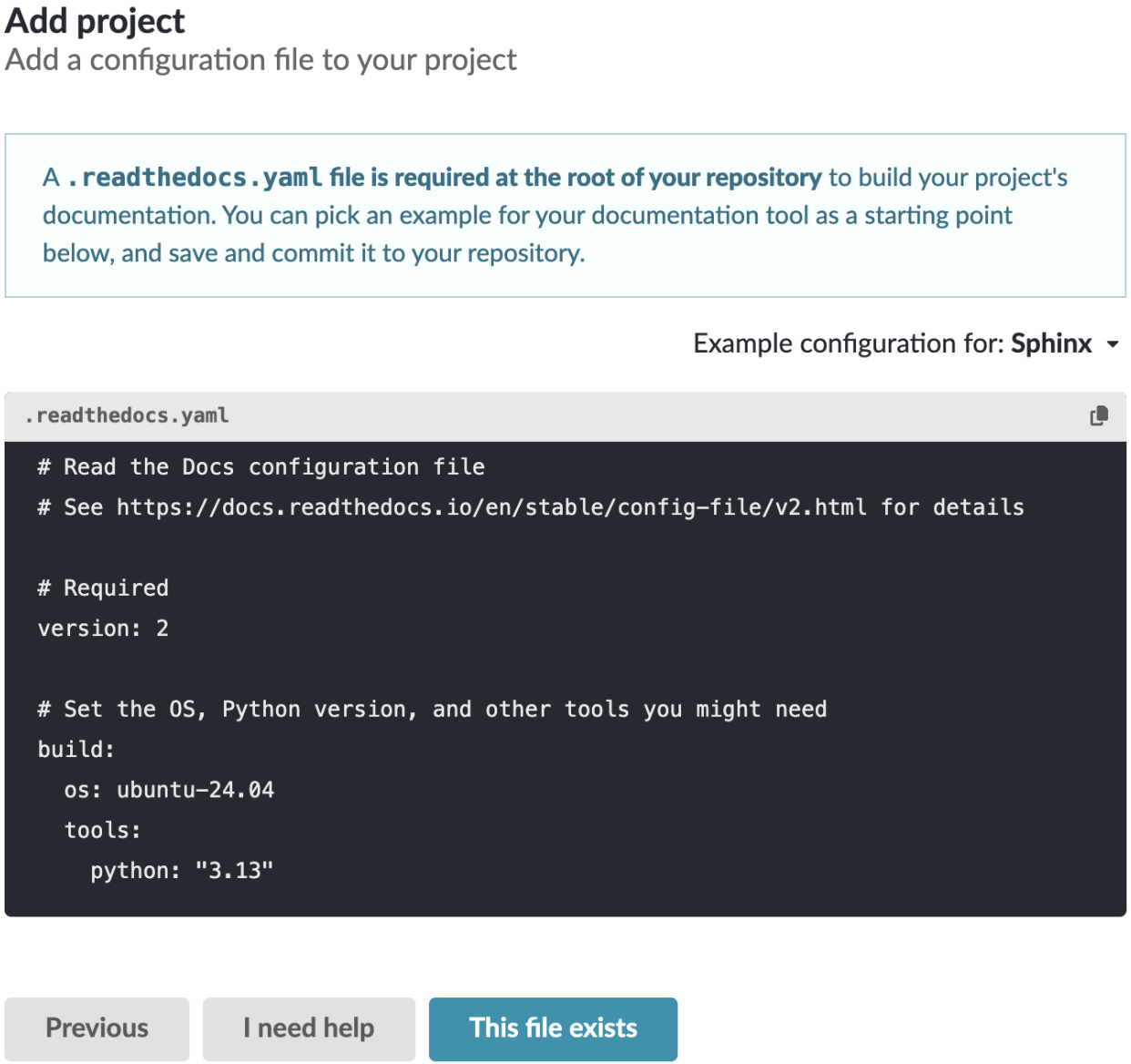


Configure settings



- Next, it should tell you that a .readthedocs.yaml file is required at the root of your repository

.readthedocs.yaml file



1. Create a .readthedocs.yaml file in your repository (top level).
2. Copy the contents of their Sphinx example (in screenshot) to your project's .readthedocs.yaml and save. **Important**: there is more text than shown in the screenshot. Copy it completely
3. Merge this change to your project's repository:

```
git add .
git commit -m ”Added readthedocs.yaml"
git push
```

4. Then select “This file exists”

Building

ReadTheDocs might error and say that some modules are missing

1. Create a new file in your docs/ folder called requirements.txt and add the following:

```
numpy
matplotlib
sphinx
sphinx_rtd_theme
myst_parser
```

2. Uncomment the following in .readthedocs.yaml :

```
python:
install:
- requirements: docs/requirements.txt
```

3. Then push your changes

```
git add .
git commit -m ”Added requirements"
git push
```

It should build successfully

View the built docs at